

PROJECT AION

Integration Brainstorm

Future Integration Strategy

Jarvis (Master Archon) + Alfred-Pro (Operations Archon)

Generated: 2026-03-29 10:58

Version 1.0.0

Jarvis v5.11.0 | Alfred-Pro v3.2.0 | Pulse v1.0.0-aion

Multi-Archon Integration Brainstorm — Project Aion v2.0

Date: 2026-03-29

Author: Jarvis (Master Archon), Session 49

Status: Strategic vision document for user review

1. Where We Are Today

We have two operational Archons connected through Pulse:

- **Jarvis** — Deep collaborative coding in persistent tmux sessions with JICM context management, 4-tier memory, 9 autonomic components. Currently focused on Chronicler (Dwarf Fortress companion app) but architecture is project-agnostic.
- **Alfred-Pro** — Always-on infrastructure automation with Nexus dispatcher (24 personas, cron-driven), Pulse task management (60+ labels), and Telegram notifications via @KeryxArchon_bot.
- **Pulse** — Shared PostgreSQL-backed task API running 24/7 via launchd, enabling cross-Archon task visibility and coordination.

What's missing: The Archons operate independently. Alfred doesn't know when Jarvis is in a deep work session. Jarvis doesn't automatically create Pulse tasks from its work. There's no unified dashboard. There's no way for a family member to interact with the system.

2. The Full Integration Vision

2.1 Aion Hub — Unified Dashboard

What: A web-based dashboard (React + FastAPI) that serves as the single pane of glass for all Archon activity.

Pages:

- **Overview:** All Archon status, active tasks, recent events, health summary
- **Tasks:** Full Pulse task board (Kanban, list, timeline views)
- **Archons:** Per-Archon detail (Jarvis session state, Alfred job status, future Archons)
- **Infrastructure:** Docker health, service ports, resource usage
- **Notifications:** Telegram message history, approval queue
- **Projects:** Cross-project status (Chronicler, personal projects, family tasks)

- **Family:** Task boards for family members (simplified, role-based access)
- **Settings:** Archon configuration, notification preferences, user profiles

Technology: Reuse Alfred's dashboard pattern (React + TypeScript + Fastify). Pulse already provides the data API. Add WebSocket for real-time updates.

Access: Local network (192.168.x.x) for family members. Authentik SSO or simple auth for multi-user.

2.2 Archon Orchestration Protocol

What: A formal protocol for how Archons coordinate, delegate, and hand off work.

Task Delegation Flow:

```
User creates task (via dashboard, Telegram, or CLI)
→ Pulse stores task with labels
→ Task Evaluator (Alfred Nexus job) scores risk + capability
→ Routes to appropriate Archon:
  agent:jarvis → Deep coding, analysis, architecture
  agent:aifred → Infrastructure, monitoring, cleanup
  agent:sentinel → Security scanning, threat detection
  agent:scholar → Research, knowledge management
  agent:shared → First available Archon
→ Archon claims task (stage:execute)
→ Work happens (in tmux session or headless)
→ Archon closes task with results
→ Event logged to Pulse audit trail
```

Jarvis Session Integration:

- AC-01 (Self-Launch) reads Pulse for agent:jarvis tasks on startup
- AC-09 (Session End) closes completed Pulse tasks, creates follow-ups
- JICM compression preserves active Pulse task IDs
- Wiggum Loop checks Pulse for new high-priority tasks between work blocks

2.3 Loom — Content Generation Pipeline

What: A model routing + content generation + delivery system for local LLM work.

Architecture:

```
Task (from Pulse, type:content-generation)
→ Model Router (select best model for task)
  ■■■ Simple (summaries, formatting): Qwen3-8B local (~0.25s)
  ■■■ Medium (analysis, research): Qwen3-30B local (~2-5s)
  ■■■ Complex (architecture, creative): Claude API (Sonnet/Opus)
  ■■■ Embeddings: Qwen3-Embedding-4B MLX local
→ Template Resolution (prompt template per content type)
→ Context Assembly (pull from Pulse, files, knowledge/)
→ Generation (selected model)
→ Quality Scoring (automated: coherence, completeness, task-alignment)
→ Iteration Loop (re-generate if score < threshold)
→ Delivery (filesystem, Telegram, dashboard, Obsidian, email)
```

Content Types:

- Research notes → Obsidian vault / knowledge base
- Session summaries → Pulse task notes
- Code documentation → Project README / inline docs
- Reports → PDF via reportlab
- Creative writing → Filesystem or Telegram
- Family updates → Telegram or dashboard

Model Registry (loom-models.yaml):

```
models:
- id: qwen3-8b
  endpoint: http://localhost:11434
  latency: ~0.25s
  cost: $0
  quality: 6/10
  best_for: [summarize, format, classify, label]
- id: qwen3-30b
  endpoint: http://localhost:11434
  latency: ~3s
  cost: $0
  quality: 8/10
  best_for: [analyze, research, draft]
- id: claude-sonnet
  endpoint: anthropic-api
  latency: ~2s
  cost: $0.003/1K
  quality: 9/10
  best_for: [code, architecture, complex-analysis]
- id: claude-opus
  endpoint: anthropic-api
  latency: ~5s
  cost: $0.015/1K
  quality: 10/10
  best_for: [creative, novel-architecture, critical-decisions]
```

2.4 Additional Archons

Sentinel — Security Archon

- Continuous secret scanning across all repos
- Docker image vulnerability checking
- Network port monitoring (unexpected listeners)
- SSL certificate expiry tracking
- File permission auditing
- Based on: Alfred's security guards + expanded scope
- Persona: security-reviewer (already exists in Alfred-Pro)
- Nexus job: weekly threat intel, daily secret scan

Scholar — Research Archon

- Deep research with citation tracking (web, academic, technical)

- Knowledge graph maintenance (Graphiti ingestion at scale)
- RAG collection curation (freshness auditing, dedup, re-indexing)
- Loom content generation pipeline owner
- Based on: Jarvis research-ops + Alfred researcher persona
- Could run on a dedicated model (larger Qwen for local inference)

Artificer — Deployment Archon

- Docker service lifecycle (create, update, rollback, health)
- VM orchestration (UTM VMs, remote servers)
- CI/CD pipeline (GitHub Actions, local build scripts)
- Backup management (Restic, Time Machine)
- Based on: Alfred infrastructure-deployer + Jarvis docker-deployer

Herald — Communication Archon (new concept)

- Multi-channel notification management (Telegram, email, SMS, dashboard)
- Message priority and DND enforcement
- Family member notification preferences
- Calendar integration
- Based on: Alfred msg-relay + expanded to support family

2.5 Docker Infrastructure Expansion

Current stack (5 containers + Pulse):

Container	Purpose	Keep/Expand
jarvis-postgres	All databases	Keep — add Aion Hub DB
jarvis-qdrant	Vector search	Keep — add per-Archon collections
jarvis-neo4j	Knowledge graph	Keep — add per-Archon group_ids
jarvis-redis	Cache	Keep — add pub/sub for Archon events
jarvis-n8n	Workflows	Expand — Archon coordination workflows

New containers to consider:

Container	Purpose	Priority
aion-hub	Dashboard (React + Fastify)	High — unified user interface
aion-loom	Content generation pipeline	Medium — after Loom design
aion-sentinel	Security scanner	Low — can run as Nexus job initially
grafana	Metrics visualization	Medium — Nexus already exports Prometheus

loki	Log aggregation	Medium — Nexus already produces JSON logs
------	-----------------	---

2.6 Unified Script Architecture

Current scripts are scattered:

- Jarvis: `.claude/scripts/` (65 scripts, tmux-focused)
- Alfred-Pro: `scripts/` (30+ scripts, service management) + `.claude/jobs/lib/` (14 support scripts)

Proposed unification:

```
/Users/nathanielcannon/Claude/Aion/
bin/                                # Unified CLI entry points
  aion                             # Master CLI (dispatches to sub-commands)
  aion-pulse                        # Pulse task management
  aion-nexus                        # Nexus job management
  aion-health                       # System health check
  aion-archon                      # Archon management (start/stop/status)
lib/                                # Shared libraries
  shared.sh                        # Bash utilities (from AIfred hooks/lib/shared.js)
  pulse-api.sh                    # Pulse API helpers (from AIfred jobs/lib/pulse-api.sh)
  telegram.sh                     # Telegram helpers
config/                             # Centralized configuration
  archons.yaml                    # Archon registry (name, role, location, status)
  services.yaml                  # Service registry (ports, health checks)
  models.yaml                    # LLM model registry (Loom)
```

2.7 Hook Architecture Consolidation

Both Jarvis and Alfred have extensive hook systems. For the multi-Archon model:

Shared hooks (applicable to all Claude Code sessions in the Aion ecosystem):

- `audit-logger.js` → Unified audit trail to Pulse events API
- `branch-protection.js` → Consistent across all repos
- `credential-guard.js` → Centralized secret detection
- `session-start.js` → Pulse task check on every session start

Archon-specific hooks (stay in their respective repos):

- Jarvis: JICM hooks, insight-capture, ulfhedthnar-detector, virgil-tracker
- Alfred: document-guard (4-tier), persona-guard, docker-validator

Implementation: Shared hooks in a common location (symlinked or npm-linked). Each Archon's `settings.json` references both shared and local hooks.

3. Family & Life Management

3.1 Family Task Board

Concept: A simplified Pulse interface for family members. Each person gets:

- Their own label (agent:nathaniel, agent:spouse, agent:child)
- A Telegram bot for task creation ("Hey Keryx, remind me to...")
- A dashboard view filtered to their tasks
- Notification preferences (DND hours, urgency thresholds)

Task types for family:

- Reminders and to-dos
- Grocery/shopping lists (shared, collaborative)
- Calendar events (sync'd via CalDAV or Google Calendar API)
- Home maintenance schedule
- Bill payments and financial tasks
- Travel planning

3.2 Personal AI Assistant (via Telegram)

Concept: @KeryxArchon_bot becomes a multi-purpose assistant:

- Natural language task creation → Pulse task with auto-labels
- Status queries ("What's on my plate today?")
- Approval workflows ("This Docker update is ready — approve?")
- Research requests → Scholar Archon generates answer, delivers via Telegram
- Home automation triggers (if integrated with Home Assistant)
- Weather, calendar, reminders

Implementation: Expand telegram-callback-handler.sh to parse natural language, create Pulse tasks with appropriate routing labels. Use Loom for response generation.

3.3 Professional Task Management

Concept: Pulse handles professional work alongside personal:

- Labels: domain:professional, project:work-client-A, etc.
 - Separate notification channel for work tasks
 - Time tracking (claim/close timestamps → work hours calculation)
 - Deliverable tracking (link tasks to git commits, documents)
 - Weekly summary reports (Loom-generated, delivered via email/Telegram)
-

4. Technical Deep Dives

4.1 Jarvis ↔ Alfred State Synchronization

Problem: Jarvis's session-state.md and Alfred's Pulse tasks need to stay in sync.

Solution:

1. AC-01 (Jarvis startup): Query Pulse for agent:jarvis tasks, incorporate into session priorities
2. Work completion: Jarvis closes Pulse tasks and updates session-state.md
3. JICM compression: Active Pulse task IDs preserved in compressed context
4. Alfred's doc-sync-check job: Monitors session-state.md for Jarvis activity

4.2 Shared Knowledge Graph

Current: Neo4j with single group_id jarvis-core

Proposed: Multiple group_ids for knowledge partitioning:

- jarvis-core — Jarvis session knowledge, architectural decisions
- aifred-ops — Infrastructure state, health history, job outcomes
- chronicler — Dwarf Fortress domain knowledge
- aion-shared — Cross-Archon knowledge (integration patterns, lessons)

Each Archon writes to its own group_id but can read from all. Graphiti's temporal awareness means stale knowledge naturally ages out.

4.3 Model Routing Intelligence

Current: Manual model selection via capability-map.yaml (opus/sonnet/haiku tiers)

Proposed: Unified model routing for all Archons:

1. Task arrives with capability:* label
2. Model router checks: local model available? Sufficient quality? Within budget?
3. Routes to cheapest model that meets quality threshold
4. Falls back to API models for complex/critical tasks
5. Tracks cost per model per task type → optimizes routing over time

4.4 Event-Driven Architecture

Current: Cron-based (5-min poll) + signal files (file-based triggers)

Proposed: Redis pub/sub for real-time Archon events:

- Channel: aion:events — all Archon events

- Channel: `aion:tasks:{archon}` — task assignments per Archon
- Channel: `aion:health` — health check results
- Fallback: Cron still runs as safety net (catches missed events)

Redis is already running (`jarvis-redis`). Adding pub/sub is zero-infrastructure cost.

5. Implementation Priorities

Immediate (This Week)

Priority	Task	Complexity
1	Wire Jarvis AC-01 to read Pulse tasks on startup	Medium
2	Wire Jarvis AC-09 to close Pulse tasks on session end	Medium
3	Test Nexus dispatcher end-to-end (force-run health-check job)	Low
4	Configure Nexus job registry for Mac Studio services	Medium

Near-Term (This Month)

Priority	Task	Complexity
5	Build Aion Hub dashboard (React + Fastify, basic views)	High
6	Design Loom model routing YAML spec	Medium
7	Implement Sentinel security scanning (as Nexus job first)	Medium
8	Family task board (Telegram natural language → Pulse tasks)	Medium
9	Hook consolidation (shared hooks across Archons)	Medium

Medium-Term (This Quarter)

Priority	Task	Complexity
10	Full Loom content generation pipeline	High
11	Scholar Archon (dedicated research agent)	High

12	Aion unified CLI (aion command)	Medium
13	Redis pub/sub for real-time Archon events	Medium
14	Multi-user auth (family members)	Medium
15	Professional task management with time tracking	Medium

Long-Term (Future)

Priority	Task	Complexity
16	Artificer Archon (deployment automation)	High
17	Herald Archon (multi-channel communications)	High
18	Home automation integration (Home Assistant)	High
19	Mobile app (React Native, connects to Pulse API)	High
20	Multi-machine Archon distribution (Mac Studio + HomeServer)	Very High

6. Architectural Principles for Multi-Archon

- 1. Shared nothing, shared everything:** Archons share infrastructure (PostgreSQL, Qdrant, Neo4j, Redis) but own their code, config, and identity. No Archon modifies another's files.
- 2. Pulse is the nervous system:** All inter-Archon communication flows through Pulse tasks and events. No direct file-based coupling between Archons.
- 3. Labels are the language:** The label taxonomy is the vocabulary all Archons speak. Extending the taxonomy extends system capability.
- 4. Personas are sandboxes:** Every headless job runs with explicit permissions. No job gets more than it needs.
- 5. Human at the top:** Critical, destructive, or uncertain actions always escalate to human approval (via Telegram or dashboard). The system proposes, the human disposes.
- 6. Learn from feedback:** Every Archon captures corrections and feedback. The AI David learning loop (agreed/wrong/adjust) applies to all Archons over time.
- 7. Cost-aware execution:** Pre-check gates before LLM invocation. Local models when possible. API models when necessary. Track and report costs.
- 8. 24/7 infrastructure, on-demand intelligence:** Infrastructure services (Pulse, Docker, databases) run always. AI inference runs only when triggered by schedule, signal, or human request.

7. What Makes This Different

Most AI automation frameworks are either:

- **Single-agent** (one Claude Code session doing everything)
- **Swarm-based** (many disposable agents with no persistent identity)
- **Platform-dependent** (tied to a specific cloud service)

Project Aion is:

- **Multi-Archon**: Named, specialized agents with persistent identity and memory
- **Self-hosted**: Everything runs on a single Mac Studio (expandable to multiple machines)
- **Cooperative**: Archons share a task bus and knowledge graph, not just a message queue
- **Self-improving**: Each Archon reflects, learns, and evolves (AC-05/06/07)
- **Human-centered**: Designed for a family, not a corporation. Tasks include groceries and game companions alongside code deployments
- **Infrastructure-native**: Docker, PostgreSQL, vector DBs, knowledge graphs — not wrappers around API calls

This is a personal AI operating system, not a chatbot.

Multi-Archon Integration Brainstorm — Project Aion v2.0

Jarvis (Master Archon) — 2026-03-29